ENSAE 1A - Projet de Programmation 2026

Emmanuelle Chocron - Samuel Placek

Ce dépôt contient le code source du projet de programmation 2026 de l'ENSAE. L'objectif est de modéliser le déplacement d'un agent dans un environnement contraint par la fatigue et de trouver des chemins optimaux.

Partie 1 - Chemin de temps minimal

Partie 1.1 : Cas sans fatigue

Cette première étape du projet se concentre sur la recherche du chemin le plus rapide entre un point de départ et un point d'arrivée, en considérant un environnement où le coefficient de fatigue est nul ($f(e) = 0$) .

Choix d'implémentation et architecture du programme

Conformément aux spécifications, nous avons structuré le code autour de deux classes principales :

1. **Classe** `Network` :
    - Représente l'environnement complet avec les routes, les longueurs et les coefficients de fatigue.
    - **Méthode ajoutée** : `build_simple_graph(self)`. Cette méthode permet de construite un graphe, qui est un cas particulier de network où il n'y a pas de fatigue. Cela permet de se ramener à un problème de graphe pondéré classique.
2. **Classe** `Graph` :
    - Représente la structure mathématique du graphe (noeuds et arêtes pondérées).
    - **Méthode ajoutée** : `shortest_path(self, start, end)`. Cette méthode implémente l'**algorithme de Dijkstra** qui permet d'obtenir le plus court chemin.

Pourquoi l'algorithme de Dijkstra ?

Puisque nous travaillons dans cette partie avec des arêtes ayant des longueurs positives ($l(e) \ge 0$) et sans accumulation de fatigue, le problème se réduit à la recherche d'un plus court chemin dans un graphe pondéré à poids positifs. L'algorithme de Dijkstra est la solution standard la plus efficace pour ce cas de figure.

Complexité de l'algorithme

La compléxité de l'algorithme `shortest_path` est $\mathcal{O}((n+m)log(n))$ avec n le nombre de sommets et m d'arrêtes. En effet, pour l'extraction du tas, chaque noeud est visité au plus une fois et chaque heappop coûte O(log(n)). Heappop étant appelé au plus n fois, le coût total de l'extraction des tas est $\mathcal{O}(nlog(n))$. Pour les insertions dans le tas, la lige avec heappush est exécutée une fois par arête au maximum. Chaque heappush coûte $\mathcal{O}(log(n))$ et dans le pire cas le tas contient $\mathcal{O}(m)$ éléments, donc le coût total de l'insertion dans le tas est $\mathcal{O}(mlog(n))$.

Partie 1.2 : Cas avec fatigue

On considère maintenant le cas général. Il s'agit alors de chercher un plus court chemin dans un graphe étendu. Pour cela, nous avons d'abord représenté le graphe étendu dans la classe Graph. Une fois ce graphe étendu crée, il est possible de réutiliser l'algortihme de Dijskstra codé à la question précèdente.

1. Fonction `build_extended_graph` dans la classe `Network`: Le principe clé est le suivant : chaque nœud du graphe original u devient une paire (u, fatigue) dans le graphe étendu. Cela permet de modéliser le fait que passer par le même carrefour avec une fatigue différente n'est pas la même situation.

   Nous avons fait en sorte que la file queue pilote un parcours en largeur (BFS). En effet, le choix du BFS garantit qu'on explore les états par couches croissantes de fatigue, ce qui est cohérent avec la nature cumulative de la fatigue.

   Dernière remarque sur cette fonction : ce que nous avons appelé super_node constituent des nœuds d'arrivée canoniques. Comme la fatigue à l'arrivée est imprévisible (elle dépend du chemin emprunté), on crée un nœud "collecteur" (u, 0) vers lequel tous les états (u, *) convergent gratuitement (coût 0). Cela permet d'appeler `shortest_path(start, (destination, 0))` sans connaître la fatigue finale.

2. On reprend la fonction `shortest_path(self, start, end)` dans la classe `Graph`: L'intérêt d'avoir construit juste avant un graphe étendu est de pouvoir utiliser la fonction `shortest_path(self, start, end)`exactement comme elle a été implémentée en 1.1. La méthode est donc la même, la seule différence est qu'ici on appelle `shortest_path` avec un graphe étendu et non un graphe simple.

Compléxité de l'algorithme

Nous avons déjà déterminé la complexité de `shortest_path(self, start, end)` à la question précédente. Pour `build_extended_graph` : cette fonction explore le graphe étendu via un parcours en largeur (BFS). On note n le nombre de sommets du graphe original, m le nombre d'arêtes, et k le nombre d'états distincts $(u, \text{fatigue})$ explorés.

La boucle principale s'exécute k fois. À chaque itération, l'opération `queue.pop(0)` décale tous les éléments restants, coûtant $\mathcal{O}(k)$ dans le pire des cas et donc $\mathcal{O}(k^2)$ au total. De plus, la vérification `next_state not in visited_states` parcourt linéairement une liste pouvant contenir jusqu'à k éléments, et est effectuée une fois par arête étendue, ce qui coûte au total $\mathcal{O}(mk)$. La boucle interne sur les voisins contribue pour $\mathcal{O}(mk/n)$, dominé par $\mathcal{O}(mk)$.

La complexité totale est donc : $\mathcal{O}(k^2 + mk)$

Partie 1.3 : Amélioration la solution précédente en ne créant que la partie nécessaire du graphe étendu.

Etant donné que `shortest_path` a uniquement besoin d'avoir accès aux voisins d'un noeud à chaque fois que le noeud est visité, il faut donc simplement que la méthode neighbours fonctionne. Pour ce faire, nous avons créé une sous-classe de la classe `Graph`, appelée `GraphImplicit`, qui prend en argument non pas une liste d'arêtes mais une fonction qui peut retourner les arêtes à partir d'un noeud.Nous avons fait le choix de le faire de façon générique (i.e nous avons implémenté la sous-classe GraphImplicit dans le module graph et dans le module network nous avons mis une méthode qui construit le graphe implicite pour un réseau donné). En effet, GraphImplicit étant de type graph, cela nous paraisssait plus cohérent de le mettre dans Graph. Plus précisément:

1. Dans la classe `Graph` : `GraphImplicit` est une sous-classe représentant un graphe implicite c'est-à-dire un graphe dont les nœuds ne sont pas tous listés à l'avance, mais générés à la volée via une fonction neighbors. Les nœuds sont des tuples (u, fatigue) représentant un état. Parti pris : représentation implicite, adaptée aux graphes très grands voire potentiellement infinis.

2. Dans la classe `Network` : `build_implicit_graph` construit un graphe implicite d'états étendus (nœud, fatigue). On a choisi de ne pas énumérer tous les états à l'avance car la fonction `generate_neighbors` les génère à la demande.

   Dans `generate_neighbors`, chaque nœud est un état étendu (position, fatigue accumulée) et le fait que la fatigue soit encodée dans le nœud permet de traiter le poblème comme un simple parcours de graphe.

Compléxité de l'algorithme:

Pour `build_implicit_graph` : le coût est reporté sur chaque appel à generate_neighbors. Pour un état (u, fatigue), la fonction parcourt la liste des voisins de u dans `self._roads`, soit deg(u) voisins, et construit la liste correspondante en $\mathcal{O}(deg(u))$. Sur l'ensemble d'une recherche de chemin explorant k états distincts, le coût total des appels à generate_neighbors est :

$$O\left(\sum_{(u,f) \text{ exploré}} \deg(u)\right) = O\left(k \cdot \frac{m}{n}\right)$$

ce qui dans le pire cas vaut $\mathcal{O}(mk/n)$. C'est un gain significatif par rapport à build_extended_graph qui coûtait $O(k^2 + m \cdot k)$ en raison de ses structures de données inadaptées.

Bonus : représentation graphique du graphe et du chemin de temps minimal.

Pour pouvoir générer les images des réseaux, ce projet utilise **Graphviz**. Si vous souhaitez utiliser la fonctionnalité de rendu visuel du graphe, vous devez installer `graphviz`.

1. Installation du paquet Python : Ouvrez votre terminal et exécutez :

```
pip install graphviz
```

2. Installation du logiciel système : Le paquet Python n'est qu'une interface. Le logiciel Graphviz doit être installé sur votre machine :

- **Windows :** Téléchargez l'installateur sur graphviz.org/download (lors de l'installation, il faut cocher la case "*Add Graphviz to the system PATH*").
- **macOS :** Ouvrez un terminal et tapez `brew install graphviz`
- **Linux (Ubuntu/Debian) :** Ouvrez un terminal et tapez `sudo apt-get install graphviz`

Nous avons commencé par créer le réseau, puis nous avons calculé le chemin optimal associé via `shortest_path`. Enfin, nous avons généré le graphe mon_reseau.

Nous avons choisi de représenter sur les arêtes le couple ($fatigue$, $longueur$) au lieu de $temps$ car sur la lecture du graphe le choix du plus court chemin pouvait paraître paradoxal du fait de l'absence de coefficient mutiplicateur (i.e la fatigue) v pour le temps.

Partie 2 - Améliorations algorithmiques

Partie 2.1 : Méthode de prunning

Pour commencer à optimiser notre programme (on va améliorer la méthode de graphe implicite), on va tout d'abord utiliser une méthode dite de "prunning" afin de retirer les noeuds Pareto dominés du graphe : si on atteint un noeud v avec une fatigue f et en un temps t alors qu'on avait déjà atteint v avec une fatigue et un temps inférieurs, alors on peut retirer l'option (v, f, t).

Pour cela, on implémente un "oracle", une fonction qui prend un paramètre un noeud $node$ et qui répond à la question suivante : "Est-ce qu'il est utile de visiter le noeud $node$?". Il suffit juste d'acutaliser une liste d'historique qui ajoute a l'historique chaque noeuds qu'on a déjà visité (ceux où l'oracle à dit oui) puis de comparer le noeud qu'on examine à ceux de l'historique.

Il ne reste alors plus qu'à modifier la méthode `shortest_path` en conséquence. Pour ne pas entrer en conflit avec les parties précédentes, comme on ne fait que travailler avec le graphe implicit, on crée une nouvelle méthode `shortest_path` de la classe `GraphImplicit` qui utilise l'oracle

Partie 2.2 : Algorithme A^*

Avant d'implémenter l'algorithme, il s'agit de déterminer une heuristique adaptée, c'est-à-dire une technique permettant de résoudre le problème plus rapidement. Pour que celle-ci soit admissible, il faut qu'elle ne surestime pas le coût réel pour atteindre la destination, soit ici le temps. Or ici, $temps$ = $longueur$ * $fatigue$. Mais la fatigue ne peut qu'augmenter : pour un prochain sommet e on aura $f(e)$ $\ge$ fatigue_actuelle. On doit donc minimiser la longueur pour atteindre l'arrivée et alors on aura t_{min} = l_{min} * fatigue_actuelle.

Pour cela, on utilise la graphe simple (le graphe sans fatigue) pour déterminer l_{min} la distance entre le noeud et l'arrivée avec la méthode de Djikstra. Cependant, la complexité sera lourde si on applique l'algorithme à chaque noeud pour avoir l_{min}. C'est pourquoi on va réimplémenter Djikstra à l'envers : on va appeler l'algorithme sur le sommet d'arrivée et cela va nous renvoyer un dictionnaire simple_dist de la forme { $node$: l } où l est la distance entre le noeud $node$ et l'arrivée. Ainsi, on aura par un seul appel à Djikstra un dictionnaire contenant toute les distances nécessaires.

Finalement, pour implémenter l'algorithme A^*, il suffit de reprendre la méthode `shortest_path` sauf que pour chosir le prochain sommet min_node, on ne va pas choisir celui qui minimime la distance mais celui qui minimise la quantité $distance$ + $heuristique$

Compléxité de l'algorithme A^*

La structure de l'algorithme est identique à celle de Dijkstra sur le graphe étendu, avec le coût supplémentaire du calcul du f-score. La boucle principale effectue au plus k extractions du tas, chacune en $\mathcal{O}(log(k))$, pour un coût total de $\mathcal{O}(k \ log(k))$. Pour chaque état extrait, on explore ses voisins via `self.neighbors` (au plus deg(u) voisins pour un état (u, fatigue))et on effectue pour chacun une insertion dans le tas en $\mathcal{O}(log(k))$. Le nombre total de voisins explorés sur l'ensemble des itérations est $\mathcal{O}(mk/n)$, ce qui donne un coût total d'insertion de $\mathcal{O}(mk/n \ log(k))$. La reconstruction du chemin via `came_from` coûte $\mathcal{O}(k)$ au plus. La complexité totale est donc :
$$O\left(\left(k + \frac{m \cdot k}{n}\right) \log(k)\right) = O\left(\frac{m \cdot k}{n} \log(k)\right)$$

Partie 3 : Extention - Le jeu à deux joueurs

L'objectif est d'atteindre le sommet d'arrivée avec le **temps total le plus faible**.

Afin de garantir la robustesse du projet et d'offrir une expérience utilisateur optimale, le moteur de jeu a été décliné en deux interfaces distinctes partageant la même logique algorithmique :

1. Version Terminal (Mode Console)

Cette implémentation est compatible avec Onyxia car elle n'ouvre pas d'onglet extérieur comme le fait `pygame`. Le principe est le suivant :

- **Boucle de jeu sécurisée :** Gestion des entrées utilisateur avec vérification stricte des sommets atteignables.
- **Affichage des états :** Suivi clair des statistiques (Position, Temps, Fatigue) à chaque tour directement dans la console.
- **Indépendance :** Ne nécessite aucune bibliothèque externe pour fonctionner.

2. Version Graphique (Mode Pygame)

Cette implémentation permet une représentation visuelle et intéractive. Le principe est le suivant :

- **Génération spatiale dynamique :** Les sommets du graphe ne possédant pas de coordonnées géographiques natives, nous avons développé un algorithme de placement sur grille avec décalage aléatoire pour répartir harmonieusement les villes sans chevauchement.
- **Rendu conditionnel :** Le code couleur du graphe s'adapte en temps réel à l'état du jeu (mise en surbrillance verte des sommets voisins accessibles, sommet d'arrivée coloré en or, décalage visuel des pions pour éviter la superposition).

Si la version Terminal fonctionne nativement avec Python, la version graphique nécessite l'installation de la bibliothèque `pygame`.

Ouvrez votre terminal à la racine du projet et exécutez :

```
pip install pygame
```